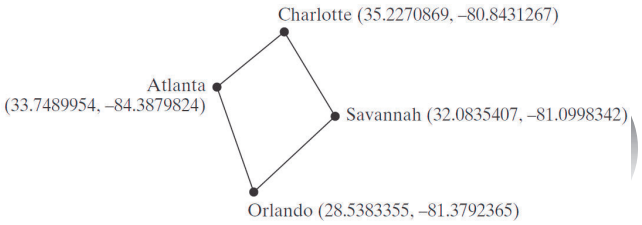
Chapter 4 Mathematical Functions, Characters, and Strings



Motivations

Suppose you need to estimate the area enclosed by four cities, given the GPS locations (latitude and longitude) of these cities, as shown in the following diagram. How would you write a program to solve this problem? You will be able to write such a program after completing this chapter.



Outline

- Mathematical Functions
- Character Data Type
- String Type



Mathematical Functions

Java provides many useful methods in the **Math** class for performing common mathematical functions.



The `Math` Class

- Class constants:
  - `PI`
  - `E`
- Class methods:
  - Trigonometric Methods
  - Exponent Methods
  - Rounding Methods
  - min, max, abs, and random Methods



Trigonometric Methods

- **`sin(double a)`**
  - `Returns the trigonometric sine of an angle in radians.`
- **`cos(double a)`**
  - `Returns the trigonometric cosine of an angle in radians.`
- **`tan(double a)`**
  - `Returns the trigonometric tangent of an angle in radians.`
- **`toRadians(degree)`**
  - `Returns the angle in radians for the angle in degree.`
- **`toDegree(radians)`**
  - `Returns the angle in degrees for the angle in radians.`
- **`acos(double a)`**
  - `Returns the angle in radians for the inverse of cosine`
- **`asin(double a)`**
  - `Returns the angle in radians for the inverse of sine.`
- **`atan(double a)`**
  - `Returns the angle in radians for the inverse of tangent`



Trigonometric Methods

Examples:

- **`Math.sin(0) returns 0.0`**
- **`Math.sin(Math.PI / 6) returns 0.5`**
- **`Math.sin(Math.PI / 2) returns 1.0`**
- **`Math.cos(0) returns 1.0`**
- **`Math.cos(Math.PI / 6) returns 0.866`**
- **`Math.cos(Math.PI / 2) returns 0`**

- **`Math.toDegrees(Math.PI /2)returns 90.0`**
- **`Math.toRadians(30)returns 0.5236(same as π/6)`**



Exponent Methods

- **`exp(double a)`**
  - Returns e raised to the power of x (e^x).
- **`log(double a)`**
  - Returns the natural logarithm of x ($\ln(x)=\log_e(x)$).
- **`log10(double a)`**
  - Returns the base 10 logarithm of x ($\log_{10}(x)$).
- **`pow(double a, double b)`**
  - Returns a raised to the power of b (a^b).
- **`sqrt(double a)`**
  - Returns the square root of x ($\sqrt{x}$) for x >=0.



Exponent Methods

Examples:

```
Math.exp(1) returns 2.71828
Math.log(Math.E) returns 1.0
Math.log10(10) returns 1.0
Math.pow(2, 3) returns 8.0
Math.pow(3, 2) returns 9.0
Math.pow(3.5, 2.5) returns 22.91765
Math.sqrt(4) returns 2.0
Math.sqrt(10.5) returns 3.24
```



Rounding Methods

- **`double ceil(double x)`**
  - x rounded up to its nearest integer. This integer is returned as a double value.
- **`double floor(double x)`**
  - x is rounded down to its nearest integer. This integer is returned as a double value.
- **`double rint(double x)`**
  - x is rounded to its nearest integer. If x is equally close to two integers, the even one is returned as a double.
- **`int round(float x)`**
  - Return `(int)Math.floor(x+0.5)`
- **`long round(double x)`**
  - Return `(long)Math.floor(x+0.5)`



Rounding Methods Examples

```
Math.ceil(2.1) returns 3.0
Math.ceil(2.0) returns 2.0
Math.ceil(-2.0) returns -2.0
Math.ceil(-2.1) returns -2.0

Math.floor(2.1) returns 2.0
Math.floor(2.0) returns 2.0
Math.floor(-2.0) returns -2.0
Math.floor(-2.1) returns -3.0

Math.rint(2.1) returns 2.0
Math.rint(2.0) returns 2.0
Math.rint(-2.0) returns -2.0
Math.rint(-2.1) returns -2.0
Math.rint(2.5) returns 2.0
Math.rint(-2.5) returns -2.0

Math.round(2.6f) returns 3
Math.round(2.0) returns 2
Math.round(-2.0f) returns -2
Math.round(-2.6) returns -3
```



min, max, and abs

- **`max(a, b)`** and **`min(a, b)`**
  Returns the maximum or minimum of two parameters.
- **`abs(a)`**
  Returns the absolute value of the parameter.
- **`random()`**
  Returns a random `double` value in the range [0.0, 1.0).

Examples:

```
Math.max(2, 3) returns 3
Math.max(2.5, 3) returns 3.0

Math.min(2.5, 3.6) returns 2.5

Math.abs(-2) returns 2
Math.abs(-2.1) returns 2.1
```



The random Method

Generates a random double value greater than or equal to 0.0 and less than 1.0 (0 <= Math.random() < 1.0).

Examples:

`(int)(Math.random() * 10)` → Returns a random integer between 0 and 9.

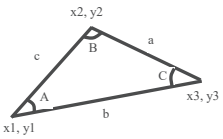
`50 + (int)(Math.random() * 50)` → Returns a random integer between 50 and 99.

In general,

`a + Math.random() * b` → Returns a random number between a and a + b, excluding a + b.



Case Study: Computing Angles of a Triangle




```
A = acos((a * a - b * b - c * c) / (-2 * b * c))
B = acos((b * b - a * a - c * c) / (-2 * a * c))
C = acos((c * c - b * b - a * a) / (-2 * a * b))
```

Write a program that prompts the user to enter the x- and y-coordinates of the three corner points in a triangle and then displays the triangle's angles.



ComputeAngles Run

Character Data Type

A character data type represents a single character.

```
char letter = 'A';
char numChar = '4';
```

A character literal is enclosed in single quotation marks



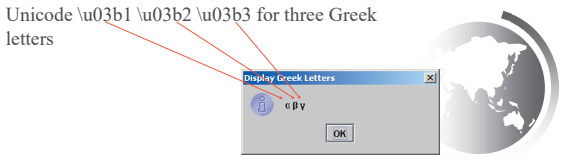
Unicode and ASCII

- Mapping a character to its binary representation is called *encoding*
  - ✓ Unicode
  - ✓ ASCII (American Standard Code for Information Interchange)



Unicode Format

Java characters use *Unicode*, a 16-bit encoding scheme established by the Unicode Consortium to support the interchange, processing, and display of written texts in the world's diverse languages. Unicode takes two bytes, preceded by \u, expressed in four hexadecimal numbers that run from '\u0000' to '\uFFFF'. So, Unicode can represent `65535 + 1 characters`.



Note: Hexadecimal Numbers

- A Hexadecimal number is based on the number **16**
- There are **16** Hexadecimal digits. They are the same as the decimal digits up to 9, but then there are the letters A, B, C, D, E and F in place of the decimal numbers 10 to 15.

Converting Hexadecimal to Binary

Hexadecimal	*Binary*	*Decimal*
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
A	1010	10
B	1011	11
C	1100	12
D	1101	13
E	1110	14
F	1111	15

ASCII Code

An 8-bit encoding scheme for representing all uppercase and lowercase letters, digits, punctuation marks, and control characters.

Characters	Code Value in Decimal	Unicode Value
'0' to '9'	48 to 57	\u0030 to \u0039
'A' to 'Z'	65 to 90	\u0041 to \u005A
'a' to 'z'	97 to 122	\u0061 to \u007A

Character Data Type

```
char letter = 'A'; (ASCII)
char letter = '\u0041'; (Unicode) // Character A's unicode is
                                     0041
```

NOTE: The increment and decrement operators can also be used on char variables to get the next or preceding Unicode character. For example, the following statements display character **b**.

```
char ch = 'a';
System.out.println(++ch);
```


Escape Sequences for Special Characters

Java uses a special notation to represent special characters. This special notation, called an escape sequence, consists of a backslash (\) followed by a character or a combination of digits.

Escape Sequence	*Name*	*Unicode Code*	*Decimal Value*
\b	Backspace	\u0008	8
\t	Tab	\u0009	9
\n	Linefeed	\u000A	10
\f	Formfeed	\u000C	12
\r	Carriage Return	\u000D	13
\\	Backslash	\u005C	92
\"	Double Quote	\u0022	34

Escape Sequences for Special Characters (cont.)

```
System.out.println("He said \"Java is fun\"");
```

displays

He said "**Java is fun**"

Escape Character

The backslash \ is called an escape character. It is a special character . To display this character, you have to use an escape sequence \\.

```
System.out.println("\\t is a tab character");
```

displays

\t is a tab character

Casting between char and Numeric Types

A *char* can be cast into any numeric type, and vice versa.

```
int i = 'a'; // Same as int i = (int)'a';
char c = 97; // Same as char c = (char)97;
```

Casting between char and Numeric Types

All numeric operators can be applied to char operands

```
int i = '2' + '3';              // (int)'2' is 50 and (int)'3' is 51
System.out.println("i is "+ i);   // i is 101

int j = 2 + 'a';                // (int)'a' is 97
System.out.println("j is "+ j);   // j is 99
System.out.println(j + " is the Unicode for character "  + (char)j);  // 99 is the Unicode
                                                                       for character c
System.out.println("Chapter "+ '2');
```

display
i is 101
j is 99
99 is the Unicode for character c
Chapter 2



Comparing and Testing Characters

Two characters can be compared using the relational operators just like comparing two numbers. This is done by comparing the Unicodes of the two characters.
For example,

- '**a**' < '**b**' is true because the Unicode for '**a**' (**97**) is less than the Unicode for '**b**' (**98**).
- '**a**' < '**A**' is false because the Unicode for '**a**' (**97**) is greater than the Unicode for '**A**' (**65**).
- '**1**' < '**8**' is true because the Unicode for '**1**' (**49**) is less than the Unicode for '**8**' (**56**).



Comparing and Testing Characters

The following code tests whether a character **ch** is an uppercase letter, a lowercase letter, or a digital character.

```
if (ch >= 'A' && ch <= 'Z')
  System.out.println(ch + " is an uppercase letter");
else if (ch >= 'a' && ch <= 'z')
  System.out.println(ch + " is a lowercase letter");
else if (ch >= '0' && ch <= '9')
  System.out.println(ch + " is a numeric character");
```



Methods in the Character Class

Java provides the following methods in the **Character** class for testing characters

Method	Description
`isDigit(ch)`	Returns true if the specified character is a digit.
`isLetter(ch)`	Returns true if the specified character is a letter.
`isLetterOfDigit(ch)`	Returns true if the specified character is a letter or digit.
`isLowerCase(ch)`	Returns true if the specified character is a lowercase letter.
`isUpperCase(ch)`	Returns true if the specified character is an uppercase letter.
`toLowerCase(ch)`	Returns the lowercase of the specified character.
`toUpperCase(ch)`	Returns the uppercase of the specified character.



Methods in the Character Class cont.

```
System.out.println("isDigit('a') is "+ Character.isDigit('a'));
System.out.println("isLetter('a') is "+ Character.isLetter('a'));
System.out.println("isLowerCase('a') is "+ Character.isLowerCase('a'));
System.out.println("isUpperCase('a') is "+ Character.isUpperCase('a'));
System.out.println("toLowerCase('T') is "+ Character.toLowerCase('T'));
System.out.println("toUpperCase('q') is "+ Character.toUpperCase('q'));
```

displays
isDigit('a') is false
isLetter('a') is true
isLowerCase('a') is true
isUpperCase('a') is false
toLowerCase('T') is t
toUpperCase('q') is Q



The String Type

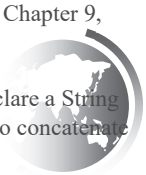
The char type only represents one character. To represent a string of characters, use the data type called *String*.
For example,

```
String message = "Welcome to Java";
```



The String Type (cont.)

- String is actually a predefined class in the Java library just like the *System* class and *Scanner* class.
- The String type is not a primitive type. It is known as a *reference type*.
- Any Java class can be used as a reference type for a variable.
- Reference data types will be thoroughly discussed in Chapter 9, "Objects and Classes."
- For the time being, you just need to know how to declare a String variable, how to assign a string to the variable, how to concatenate strings, and to perform simple operations for strings.



Simple Methods for **String** Objects

Method	Description
`length()`	Returns the number of characters in this string.
`charAt(index)`	Returns the character at the specified index from this string.
`concat(s1)`	Returns a new string that concatenates this string with string s1.
`toUpperCase()`	Returns a new string with all letters in uppercase.
`toLowerCase()`	Returns a new string with all letters in lowercase.
`trim()`	Returns a new string with whitespace characters trimmed on both sides.



Simple Methods for **String** Objects

- Strings are objects in Java.
- The methods in the preceding table can only be invoked from a specific string instance.
  - For this reason, these methods are called *instance methods*.
- A non-instance method is called a *static method*.
  - A static method can be invoked without using an object.
  - All the methods defined in the **Math** class are static methods. They are not tied to a specific object instance.
- The syntax to invoke an instance method is **referenceVariable.methodName(arguments)**.

Getting String Length

- Use the **length()** method to return the number of characters in a string.

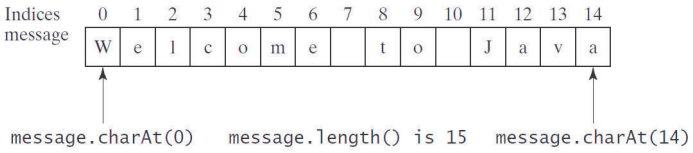
```
String message = "Welcome to Java";
System.out.println("The length of " +
message + " is " + message.length());
```

displays

The length of Welcome to Java is 15

Getting Characters from a String

The **s.charAt(index)** method can be used to retrieve a specific character in a string **s**, where the index is between **0** and **s.length()–1**.




```
String message = "Welcome to Java";
System.out.println("The first character in message is "
   + message.charAt(0));
```

displays

The first character in message is W

String Concatenation

You can use the **concat** method to concatenate two strings.

```
String s3 = s1.concat(s2);
String s3 = s1 + s2;

// Three strings are concatenated
String message = "Welcome " + "to " + "Java";

// String Chapter is concatenated with number 2
String s = "Chapter" + 2; // s becomes Chapter2

// String Supplement is concatenated with character B
String s1 = "Supplement" + 'B'; // s1 becomes SupplementB
```


String Concatenation (cont.)

The augmented += operator can also be used for string concatenation.

For example, the following code appends the string "**and Java is fun**" with the string "**Welcome to Java**" in message.

```
String message = "Welcome to Java";
message += " and Java is fun";
```


Converting Strings

The toLowerCase()method returns a new string with all lowercase letters and the toUpperCase() method returns a new string with all uppercase letters.

For example,

"Welcome".toLowerCase() returns a new string, welcome.

"Welcome".toUpperCase() returns a new string, WELCOME.

" Welcome ".trim() returns a new string, Welcome.

Reading a String from the Console

To read a string from the console, invoke the next() method on a Scanner object.

```
Scanner input = new Scanner(System.in);
System.out.print("Enter three words separated by spaces: ");
String s1 = input.next();
String s2 = input.next();
String s3 = input.next();
System.out.println("s1 is " + s1);
System.out.println("s2 is " + s2);
System.out.println("s3 is " + s3);
```

```
Enter three words separated by spaces: Welcome to Java ↵Enter
s1 is Welcome
s2 is to
s3 is Java
```


Reading a String from the Console (cont.)

The next() method reads a string that ends with a whitespace character. You can use the nextLine() method to read an entire line of text.

```
Scanner input = new Scanner(System.in);
System.out.println("Enter a line: ");
String s = input.nextLine();
System.out.println("The line entered is " + s);
```

```
Enter a line: Welcome to Java ↵Enter
The line entered is Welcome to Java
```

Important Caution

To *avoid input errors*, do not use **nextLine()** after **nextByte()**, **nextShort()**, **nextInt()**, **nextLong()**, **nextFloat()**, **nextDouble()**, or **next()**.

The reasons will be explained in Section 12.11.4, 'How Does **Scanner** Work?

Reading a Character from the Console

To read a character from the console, use the nextLine() method to read a string and then invoke the charAt(0) method on the string to return a character.

For example,

```
Scanner input = new Scanner(System.in);
System.out.print("Enter a character: ");
String s = input.nextLine();
char ch = s.charAt(0);
System.out.println("The character entered is " + ch);
```


Comparing Strings

Method	Description
`equals(s1)`	Returns true if this string is equal to string `s1`.
`equalsIgnoreCase(s1)`	Returns true if this string is equal to string `s1`; it is case insensitive.
`compareTo(s1)`	Returns an integer greater than `0`, equal to `0`, or less than `0` to indicate whether this string is greater than, equal to, or less than `s1`.
`compareToIgnoreCase(s1)`	Same as `compareTo` except that the comparison is case insensitive.
`startsWith(prefix)`	Returns true if this string starts with the specified prefix.
`endsWith(suffix)`	Returns true if this string ends with the specified suffix.

Comparing Strings (cont.)

The == operator checks only whether **string1** and **string2** refer to the same object; it does not tell you whether they have the same contents.

```
if (string1 == string2)
  System.out.println("string1 and string2 are the same object");
else
  System.out.println("string1 and string2 are different objects");
```

Use the **equals** method. The following code can be used to compare two strings:

```
if (string1.equals(string2))
  System.out.println("string1 and string2 have the same contents");
else
  System.out.println("string1 and string2 are not equal");
```


Comparing Strings (cont.)

- The compareTo method can also be used to compare two strings.
  - The method returns the value **0** if **s1** is equal to **s2**,
  - a value less than **0** if **s1** is lexicographically (i.e., in terms of Unicode ordering) less than **s2**,
  - a value greater than **0** if **s1** is lexicographically greater than **s2**.

✓ Syntax errors will occur if you compare strings by using relational operators >, >=, <, or <=.
Instead, you have to use **s1.compareTo(s2)**

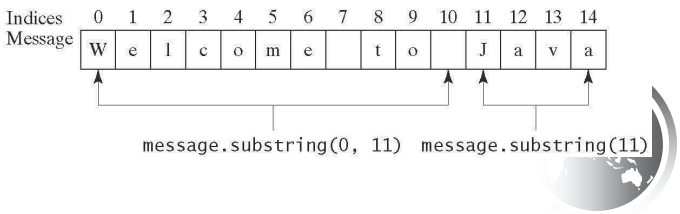
OrderTwoCities Run

Comparing Strings (cont.)

```
"Welcome to Java".startsWith("We") returns true.
"Welcome to Java".startsWith("we") returns false.
"Welcome to Java".endsWith("va") returns true.
"Welcome to Java".endsWith("v") returns false.
"Welcome to Java".contains("to") returns true.
"Welcome to Java".contains("To") returns false.
```


Obtaining Substrings

Method	Description
`substring(beginIndex)`	Returns this string's substring that begins with the character at the specified `beginIndex` and extends to the end of the string, as shown in Figure 4.2.
`substring(beginIndex, endIndex)`	Returns this string's substring that begins at the specified `beginIndex` and extends to the character at index `endIndex - 1`, as shown in Figure 9.6. Note that the character at `endIndex` is not part of the substring.



Obtaining Substrings (cont.)

```
String message = "Welcome to Java";
String message = message.substring(0, 11) + "HTML";
```

The string **message** now becomes **Welcome to HTML**.

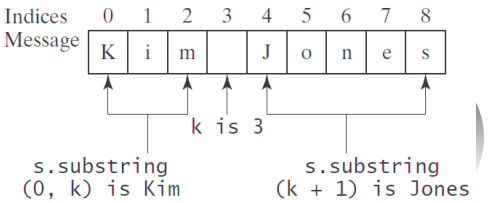
Finding a Character or a Substring in a String

Method	Description
`indexOf(ch)`	Returns the index of the first occurrence of `ch` in the string. Returns `-1` if not matched.
`indexOf(ch, fromIndex)`	Returns the index of the first occurrence of `ch` after `fromIndex` in the string. Returns `-1` if not matched.
`indexOf(s)`	Returns the index of the first occurrence of string `s` in this string. Returns `-1` if not matched.
`indexOf(s, fromIndex)`	Returns the index of the first occurrence of string `s` in this string after `fromIndex`. Returns `-1` if not matched.
`lastIndexOf(ch)`	Returns the index of the last occurrence of `ch` in the string. Returns `-1` if not matched.
`lastIndexOf(ch, fromIndex)`	Returns the index of the last occurrence of `ch` before `fromIndex` in this string. Returns `-1` if not matched.
`lastIndexOf(s)`	Returns the index of the last occurrence of string `s`. Returns `-1` if not matched.
`lastIndexOf(s, fromIndex)`	Returns the index of the last occurrence of string `s` before `fromIndex`. Returns `-1` if not matched.

Finding a Character or a Substring in a String

Suppose a string **s** contains the first name and last name separated by a space. You can use the following code to extract the first name and last name from the string:

```
int k = s.indexOf(' ');
String firstName = s.substring(0, k);
String lastName = s.substring(k + 1);
```



Conversion between Strings and Numbers

- You can convert a numeric string into a number. To convert a **string** into an **int** value, use the **Integer.parseInt** method.
- To convert a string into a double value, use the **Double.parseDouble** method.

```
String intString = "123";
int intValue = Integer.parseInt(intString);
double doubleValue = Double.parseDouble(doubleString);

int number = 2;
String s = number + "";
```


Case Study: Converting a Hexadecimal Digit to a Decimal Value

Write a program that converts a hexadecimal digit into a decimal value.

HexDigit2Dec Run

Case Study: Revising the Lottery Program Using Strings

A problem can be solved using many different approaches. This section rewrites the lottery program in Listing 3.7 using strings. Using strings simplifies this program.

LotteryUsingStrings Run

Formatting Output

Use the printf statement.

System.out.printf(format, items);

Where format is a string that may consist of substrings and format specifiers. A format specifier specifies how an item should be displayed. An item may be a numeric value, character, boolean value, or a string. Each specifier begins with a percent sign.

Frequently-Used Specifiers

Specifier	Output	Example
`%b`	a boolean value	`true` or `false`
`%c`	a character	`'a'`
`%d`	a decimal integer	`200`
`%f`	a floating-point number	`45.460000`
`%e`	a number in standard scientific notation	`4.556000e+01`
`%s`	a string	`"Java is cool"`

```
int count = 5;
double amount = 45.56;
System.out.printf("count is %d and amount is %f", count, amount);

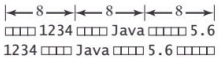
display          count is 5 and amount is 45.560000
```

items

Example

```
System.out.printf("%8d%8s%8.1f\n", 1234, "Java", 5.63);
System.out.printf("%-8d%-8s%-8.1f \n", 1234, "Java", 5.63);
```

display



where the square box (□) denotes a blank space.

FormatDemo

The example gives a program that uses **printf** to display a table.

